Assessment 2: Advanced Data Science

Handwritten Digit Recognition with a CNN

Group assignment

Student 1: Muhammadh Azwardeen Student ID: 8218414

Student 2: Diego Vertiz Padilla Student ID: 8257314

Student 3: Aadil Wazeer Student ID: 8153441

Unit code:NIT3251 Lecturer: Nafiseh

Submission date: 18/10/2026

Contents

Project overview and objectives **3**
1. CNN architecture and training **3**
1.1 CNN architecture 3
1.2 Training methodology 3
1.3 Hyperparameter experiments 3
1.4 Results and discussion 3
1.5 Limitations and contribution 3
2. Explainable AI and robustness **3**
2.1 Explainable AI methodology 3
2.2 Saliency Maps 3
2.3 Grad-CAM 3
2.4 LIME 3
2.5 Comparison of XAI techniques 3
2.6 Limitations and robustness 3
2.7 Saved outputs and contribution 3
3. Preprocessing, evaluation and error analysis **3**
3.1 Loading the trained model 3
3.2 Loading and splitting the dataset 3
3.3 Preparing the images 3
3.4 Making predictions and measuring performance 4
3.5 Confusion matrix and error analysis 5
3.6 Checking the results against the reference 6
3.7 Saved outputs and contributions 6
3.8 Limitations, improvements and conclusion 6
References **6**

Project overview and objectives

This project uses a convolutional neural network (CNN) to recognise handwritten digits from the MNIST dataset. The objectives are to train and compare models, explain predictions, measure test performance and study errors.

For evaluation, we used the selected trained model without retraining it and checked the results against the saved reference files.

1. CNN architecture and training

1.1 CNN architecture

We developed a Convolutional Neural Network (CNN) to recognise handwritten digits from the MNIST dataset. The architecture was designed to extract spatial features from grayscale images while maintaining a relatively simple structure suitable for systematic hyperparameter experiments.

The network consists of two convolutional blocks. The first applies 32 filters with a 3 × 3 kernel, ReLU activation and same padding, followed by 2 × 2 max pooling. The second uses 64 filters with the same kernel size and activation, followed by another max-pooling layer. These operations progressively extract image features while reducing the spatial dimensions from 28 × 28 to 7 × 7.

The resulting feature maps are flattened and passed into a fully connected layer with 128 neurons and ReLU activation. A final layer containing 10 neurons with softmax activation generates the probabilities for digits 0–9.

The baseline architecture contains 421,642 trainable parameters. Since a large proportion of these parameters is concentrated in the fully connected layer, we also investigated whether reducing its size could maintain competitive classification performance.

1.2 Training methodology

The original MNIST training dataset was divided into 50,000 training images and 10,000 validation images using a stratified split with random_state=42. The separate 10,000-image test dataset was reserved for final evaluation, as described in Section 3.

Images were normalised to the range 0–1 and reshaped to (28, 28, 1). All CNN configurations were trained using the Adam optimiser, sparse categorical cross-entropy loss and a batch size of 32.

To evaluate convergence and reduce unnecessary training, each model was trained for a maximum of 15 epochs with early stopping based on validation loss and a patience of three epochs. ModelCheckpoint was used to save the weights corresponding to the lowest validation loss.

The experiments followed the same training and validation procedure, allowing the effects of different hyperparameter configurations to be compared without using the test dataset for model selection.

1.3 Hyperparameter experiments

Six CNN configurations were evaluated to investigate the effects of dropout regularisation, model complexity and learning rate.

The baseline used 128 neurons in the fully connected layer with a learning rate of 0.001. A second configuration introduced dropout of 0.3 to investigate its effect on generalisation.

Two additional configurations reduced the fully connected layer to 64 and 32 neurons to evaluate the relationship between model complexity and predictive performance. Finally, learning rates of 0.0001 and 0.003 were tested using the 64-neuron architecture.

Table 5. CNN hyperparameter experiment results

Configuration	Parameters	Best epoch	Validation accuracy	Validation loss
Baseline (Dense 128)	421,642	2	98.49%	0.05480
Dropout 0.3 (Dense 128)	421,642	3	98.74%	**0.04382**
Dense 64 (LR 0.001)	220,234	2	98.67%	0.04729
Dense 32 (LR 0.001)	119,530	6	98.76%	0.04763
Dense 64 (LR 0.0001)	220,234	14	98.56%	0.05021

Dense 64 (LR 0.003)	220,234	7	**98.84%**	0.05869

Results correspond to the checkpoint with the lowest validation loss for each configuration.

1.4 Results and discussion

The dropout configuration achieved the lowest validation loss (0.04382) and a validation accuracy of 98.74%, outperforming the baseline model in both metrics. This suggests that dropout regularisation improved generalisation in this experimental run.

Reducing the fully connected layer also produced promising results. The 32-neuron configuration contained 119,530 parameters, approximately 71.7% fewer than the baseline, while achieving 98.76% validation accuracy. This indicates that a substantially smaller architecture can maintain competitive performance on MNIST, although its validation loss remained higher than that of the dropout model.

The learning rate experiments revealed differences in convergence. A learning rate of 0.0001 required 14 epochs to reach its best validation loss, whereas 0.001 reached its best result at epoch 2. The highest learning rate, 0.003, achieved the highest validation accuracy (98.84%) but also the highest validation loss (0.05869).

This demonstrates that higher classification accuracy does not necessarily correspond to lower cross-entropy loss, as the latter also considers the confidence of predicted probabilities.

Based on the predefined selection criterion of minimum validation loss, the **CNN with 128 dense neurons and dropout of 0.3** was selected as the final model.

The selected model achieved **98.86% test accuracy and a macro F1-score of 0.9886** on the held-out MNIST test set. Detailed classification metrics, the confusion matrix and error analysis are presented in Section 3.

1.5 Limitations and contribution

Although the experiments demonstrated strong performance, each configuration was trained using a single random seed. Therefore, small differences between models may reflect stochastic variation, and repeated training runs would be necessary to establish more reliable comparisons.

Additionally, the experiments focused on dropout, dense-layer capacity and learning rate while keeping the convolutional backbone unchanged. Future improvements could investigate batch normalisation, data augmentation and alternative convolutional architectures.

Overall, the experiments demonstrated that CNN performance depends not only on classification accuracy but also on regularisation, model complexity and optimisation choices. The selected trained model was saved and shared for the Explainable AI analysis described in Section 2.

Contribution: Diego Vertiz Padilla developed the CNN architecture, implemented the training pipeline, conducted the six hyperparameter experiments, compared validation results and selected the final trained model for subsequent evaluation and explainability analysis.

2. Explainable AI and robustness

2.1 Explainable AI methodology

We applied three Explainable AI (XAI) techniques to the selected CNN model to better understand how it recognised handwritten digits from the MNIST dataset. Although the model achieved 98.86% test accuracy, this result alone does not explain which image features influence its predictions.

The selected model, cnn_selected_for_xai.keras, was loaded into Google Colab. We used the MNIST test dataset, normalised the pixel values to the range 0–1 and reshaped the images to (28, 28, 1), consistent with the model's training input.

We implemented Saliency Maps, Grad-CAM and Local Interpretable Model-Agnostic Explanations (LIME). The methods were first examined using a handwritten digit 7 and then applied to one correctly classified example of each digit from 0 to 9. The trained model was not modified or retrained during this analysis.

2.2 Saliency Maps

Saliency Maps were generated using TensorFlow's GradientTape to calculate the gradient of the predicted class score with respect to the input image. We used the absolute gradient values to identify pixels where small changes could have a greater effect on the model's output.

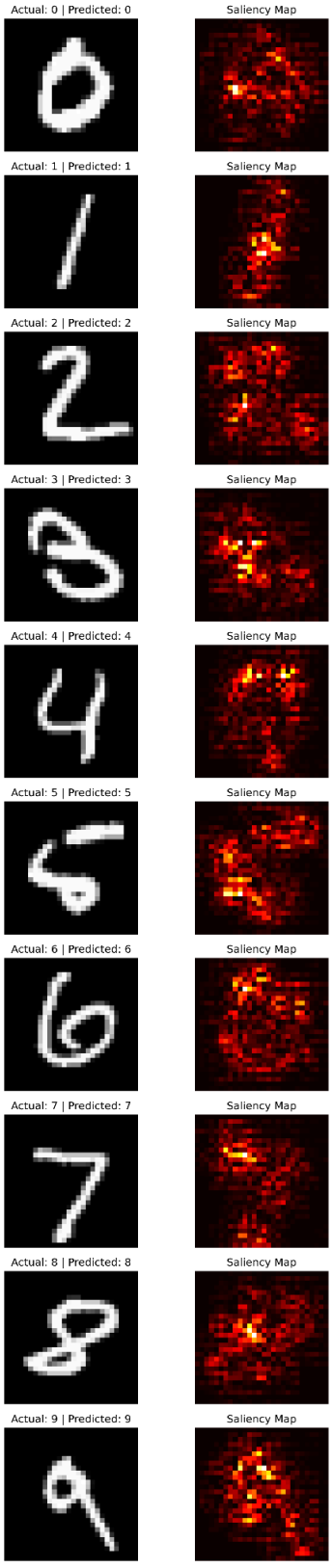


Figure 2 shows the saliency results for digits 0–9. The highlighted areas generally appeared around the handwritten strokes, although some responses were also visible in the surrounding image.

For digit 7, the strongest responses included parts of the upper horizontal stroke. This suggests that the model's prediction was sensitive to changes in these pixels. However, the saliency maps were relatively scattered, making it difficult to identify a single important region for some digits.

2.3 Grad-CAM

Gradient-weighted Class Activation Mapping (Grad-CAM) was used to examine which regions of the CNN's convolutional feature maps contributed positively to its predictions [4].

We used the output of the second convolutional layer and calculated gradients for the predicted class. These gradients were averaged to obtain weights for the feature maps, which were then combined to produce a heatmap. The resulting heatmap was resized to match the original 28 × 28 image.

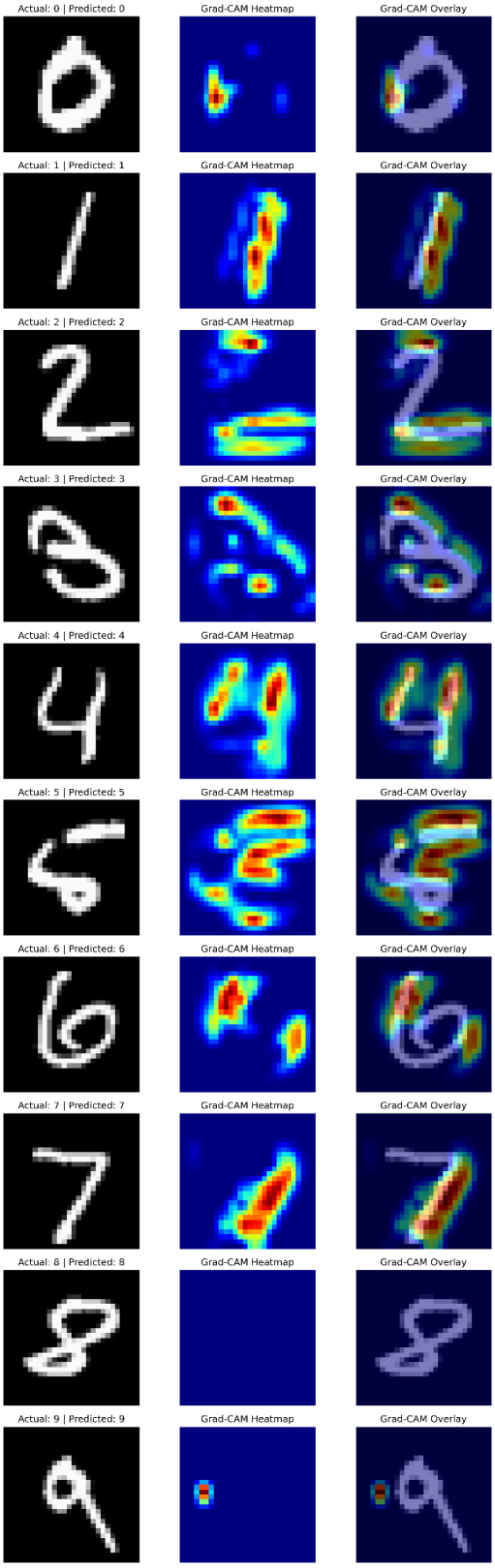
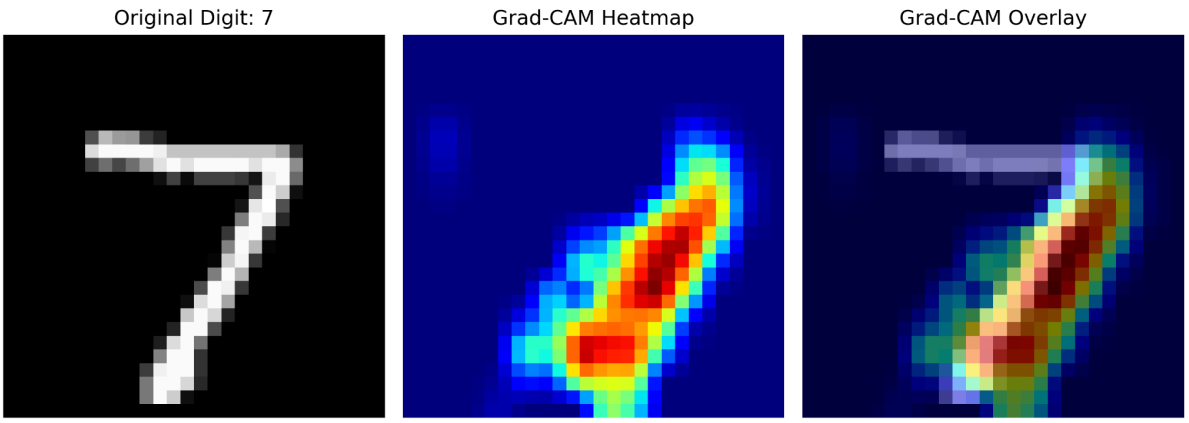


Figure 3 shows the Grad-CAM results for digits 0–9. The highlighted regions generally followed parts of the handwritten digits, but the amount of activation varied between examples.



For digit 7, shown in Figure 4, the strongest activation appeared along the diagonal stroke, particularly around the middle and lower sections. The horizontal stroke received less attention. This provided a clearer visual indication of the regions associated with the prediction than the corresponding Saliency Map.

However, Grad-CAM was less informative for some other digits. For example, the heatmap for digit 8 showed very little visible activation, despite the model predicting the digit correctly. This indicates that the quality of the visual explanation can vary between inputs.

2.4 LIME

Local Interpretable Model-Agnostic Explanations (LIME) was implemented to examine the contribution of image regions to individual predictions [5]. Unlike the gradient-based techniques, LIME explains predictions by creating variations of an input image and observing how the model responds.

We converted the grayscale images into three-channel images for LIME and used SLIC segmentation to divide each image into regions. The explanation process used 1,000 perturbed samples, with the ten selected positive features used for visualisation.

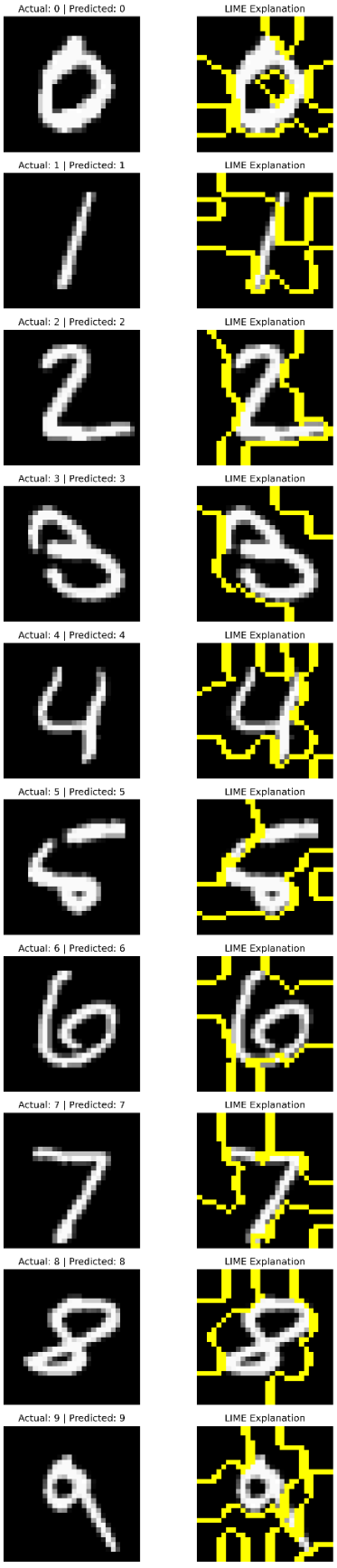
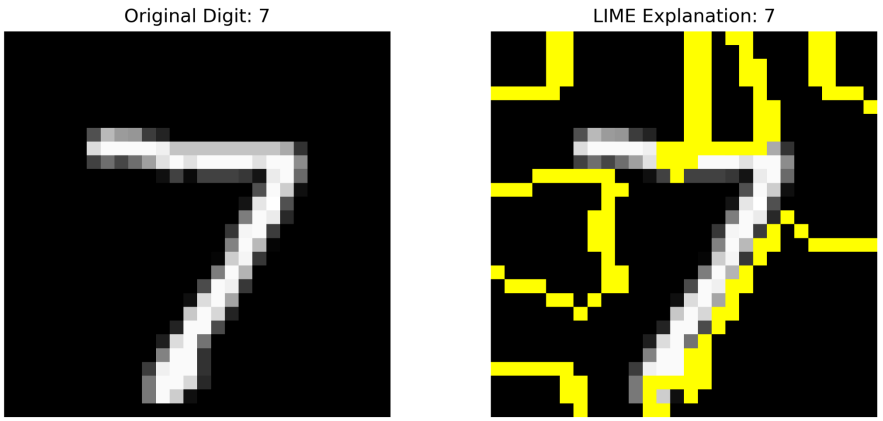


Figure 5 presents the LIME explanations for digits 0–9. Yellow boundaries identify the selected image segments associated with positive contributions to the predicted class.



For digit 7, shown in Figure 6, several highlighted segments appeared around the handwritten strokes. However, the segmentation also included parts of the background, making the explanation less straightforward to interpret. This was particularly noticeable because MNIST images are small and contain relatively simple shapes.

2.5 Comparison of XAI techniques

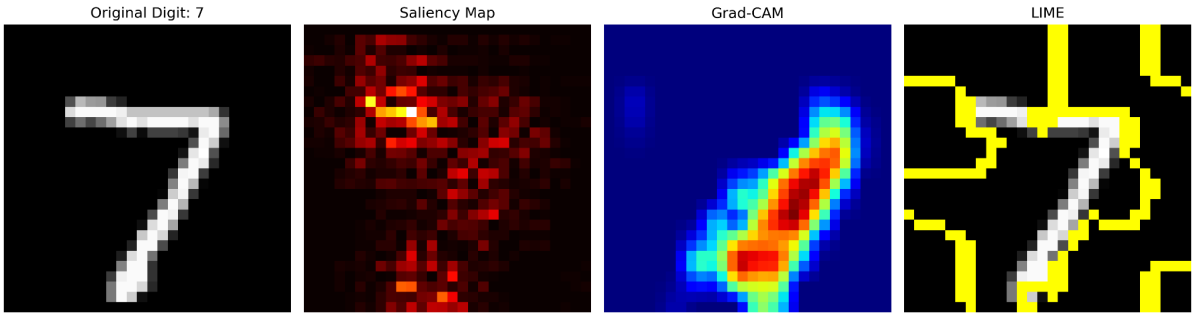


Figure 7 compares the three methods using the same handwritten digit 7.

The Saliency Map provided pixel-level information but produced scattered responses across the image. Grad-CAM generated a smoother heatmap and highlighted the diagonal stroke more clearly. LIME provided a different perspective by identifying influential image segments, although its boundaries did not always align closely with the handwritten digit.

For this particular example, Grad-CAM produced the clearest visual explanation. However, this does not establish that it is the most accurate or reliable method overall. Each technique measures a different aspect of the model's behaviour, and the results were compared visually rather than through quantitative explanation metrics.

Using the three techniques together provided a broader understanding of the CNN's predictions than relying on classification accuracy alone.

2.6 Limitations and robustness

The explanations were generated from selected MNIST test images, with one example representing each digit. Although all ten selected images were classified correctly, these

examples are not sufficient to establish how consistently the techniques explain predictions across the full dataset.

Saliency Maps can be sensitive to small input changes, while Grad-CAM depends on the selected convolutional layer. LIME also depends on the segmentation and perturbation settings, which can affect the resulting explanation.

The current analysis did not include adversarial attacks or quantitative robustness testing. Future work could introduce small image perturbations, rotations or adversarial examples to investigate whether predictions and explanations remain stable. Such experiments would help assess the model's reliability beyond correctly classified MNIST images.

2.7 Saved outputs and contribution

We saved the Saliency Map, Grad-CAM and LIME visualisations for digits 0–9, together with individual explanations and a side-by-side comparison for digit 7. The implementation was saved in explainability.ipynb, and the results were uploaded to the group's GitHub repository.

Aadil Wazeer implemented the three XAI techniques, generated and compared the visual explanations, and analysed their results and limitations.

3. Preprocessing, evaluation and error analysis

3.1 Loading the trained model

We extracted the project ZIP in Google Colab. A version mismatch prevented direct loading of cnn_selected_for_xai.keras, so we recreated the same architecture and loaded its trained weights. We confirmed the input shape (None, 28, 28, 1), output shape (None, 10) and 421,642 parameters. None allows the batch size to vary.

3.2 Loading and splitting the dataset

We loaded MNIST through Keras: 60,000 training images and 10,000 separate test images. Each image has 28 × 28 grayscale pixels and a label identifying its digit [1].

We reproduced the training and validation split using random_state=42 and stratification, which keeps similar digit proportions in both sets. The official test set remained separate. Table 1 shows the split.

Table 1. Dataset split

Set	**Images**	**Purpose**
Training	50,000	Learn model weights
Validation	10,000	Compare training choices
Test	10,000	Final evaluation

Training data teaches the model; validation data supports model selection; test data measures final performance.

3.3 Preparing the images

We converted pixels to float32 and divided by 255, scaling values to the range 0 to 1 used during training.

We added one grayscale channel, changing (N, 28, 28) to (N, 28, 28, 1). We checked shapes, data types and ranges. Preprocessing always used raw images, preventing repeated scaling when rerun. We did not apply data augmentation.

3.4 Making predictions and measuring performance

The model returned ten probabilities for each of the 10,000 test images. We used argmax to select the highest probability and compared the predicted digit with its label.

Table 2. Overall test results

Measure	**Result**
Test accuracy	98.86%
Correct predictions	9,886
Incorrect predictions	114

Macro F1	0.9886
Weighted F1	0.9886

Accuracy measures correct predictions overall. Precision measures how often predictions of a digit are correct; recall measures how many actual examples it finds. F1 combines precision and recall. Support is the number of examples. Macro F1 weights digits equally; weighted F1 accounts for support [2].

Table 3. Results for each digit (scores from 0 to 1)

Digit	**Precision**	**Recall**	**F1**	**Support**
0	0.9928	0.9908	0.9918	980
1	0.9964	0.9789	0.9876	1,135
2	0.9837	0.9971	0.9904	1,032
3	0.9767	0.9970	0.9868	1,010
4	0.9959	0.9837	0.9898	982
5	0.9887	0.9843	0.9865	892
6	0.9948	0.9906	0.9927	958
7	0.9922R	0.9864	0.9893	1,028
8	0.9887	0.9867	0.9877	974
9	0.9766	0.9911	0.9838	1,009

Accuracy was 98.86%, with 114 errors. Digit 6 had the highest F1 (0.9927); digit 9 had the lowest (0.9838).

3.5 Confusion matrix and error analysis

The confusion matrix compares actual digits in rows with predictions in columns. Diagonal entries are correct predictions; other entries are mistakes [3]. Figure 1 shows the supplied reference matrix, which our calculated matrix matched exactly.

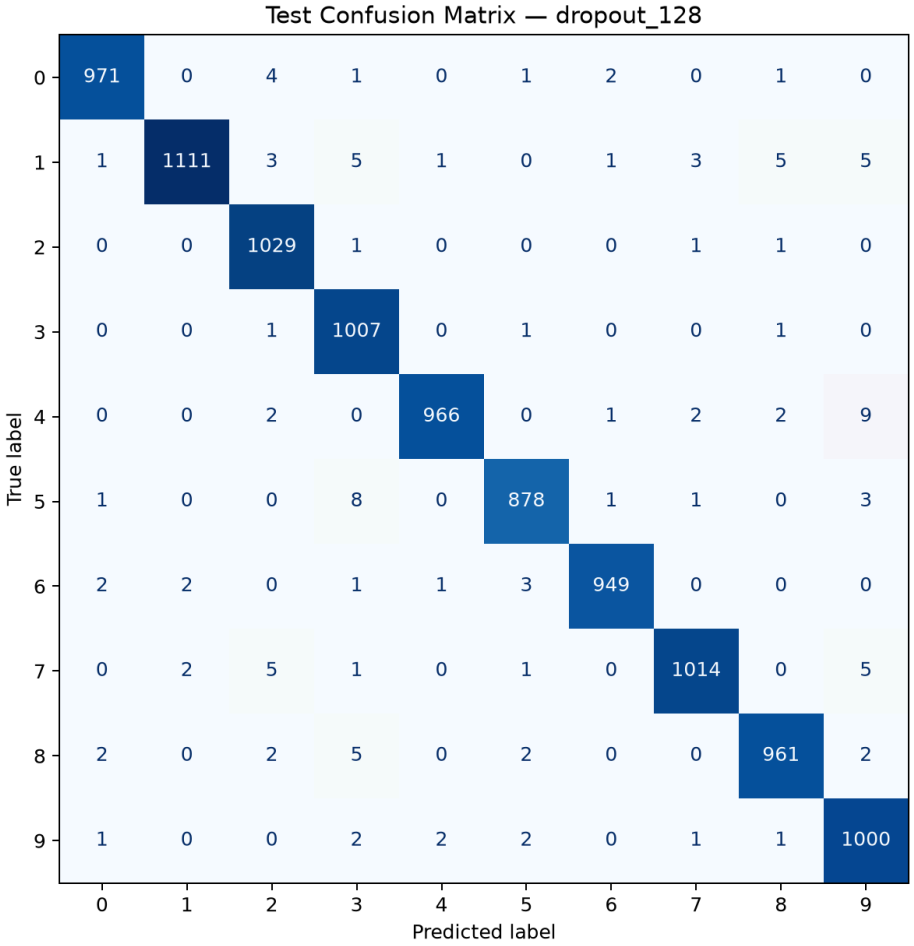


Figure 1. Test confusion matrix for the selected model. Source: supplied project reference results, verified against the Section 3 evaluation.

We ranked the mistakes by frequency. The most common pairs were 4 predicted as 9 (nine errors) and 5 predicted as 3 (eight errors).

Table 4. Most frequent confusion pairs

Actual digit	Predicted digit	Errors
4	9	9
5	3	8
1	3	5
1	8	5
1	9	5

We inspected the first 12 errors with their labels, predictions and probabilities. One 5 was predicted as 3 with 99.3% probability, showing that confidence does not guarantee correctness. Similar handwriting may explain some errors, but visual inspection alone cannot establish the model's reasoning.

3.6 Checking the results against the reference

Both reference checks returned True: np.isclose confirmed matching accuracy, and np.array_equal confirmed all 100 confusion-matrix counts. This reproduced the overall results, without checking individual output probabilities.

3.7 Saved outputs and contributions

We saved classification_report.csv, confusion_matrix.csv, ranked_confusions.csv and evaluation_summary.json. These files record the scores, confusion counts, ranked errors and overall summary.

Muhammadh Azwardeen completed data preparation, model loading for evaluation, performance measurement, error analysis, reference checks and result exports.

3.8 Limitations, improvements and conclusion

The model correctly classified 9,886 test images. Its errors included confusion between 4 and 9, and between 5 and 3. These results apply to MNIST; performance on other handwriting or phone photos remains untested.

Only 12 mistakes were visually inspected. Future work could review all 114 errors, test deliberately altered images and investigate small rotations or shifts during training. These improvements were not tested here. Future model choices should use validation data before a separate final test.

References

[1] Keras. MNIST digits classification dataset. https://keras.io/api/datasets/mnist/

[2] scikit-learn. classification_report. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html

[3] scikit-learn. confusion_matrix. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html